

DESARROLLO WEB EN ENTORNO SERVIDOR

1. INTRODUCCIÓN

- Un **servidor** es un programa que se ejecuta en una máquina conectada a una red y que permanece dormido hasta que una petición procedente de la red lo despierta. Entonces, el programa hace algo (consulta datos, elabora un cálculo, lo que sea) y devuelve su resultado por la red.

Por extensión, un servidor también es cualquier ordenador donde se ejecute un programa servidor. Es decir, usamos la misma palabra para referirnos a un programa y al ordenador donde se ejecuta ese programa.

- El **cliente** es un programa que envía peticiones al servidor para despertarlo. También es el programa que recoge el resultado devuelto por el servidor.

¿Y sabes qué? Que, por extensión, la máquina donde se ejecuta un programa cliente también se llama cliente.

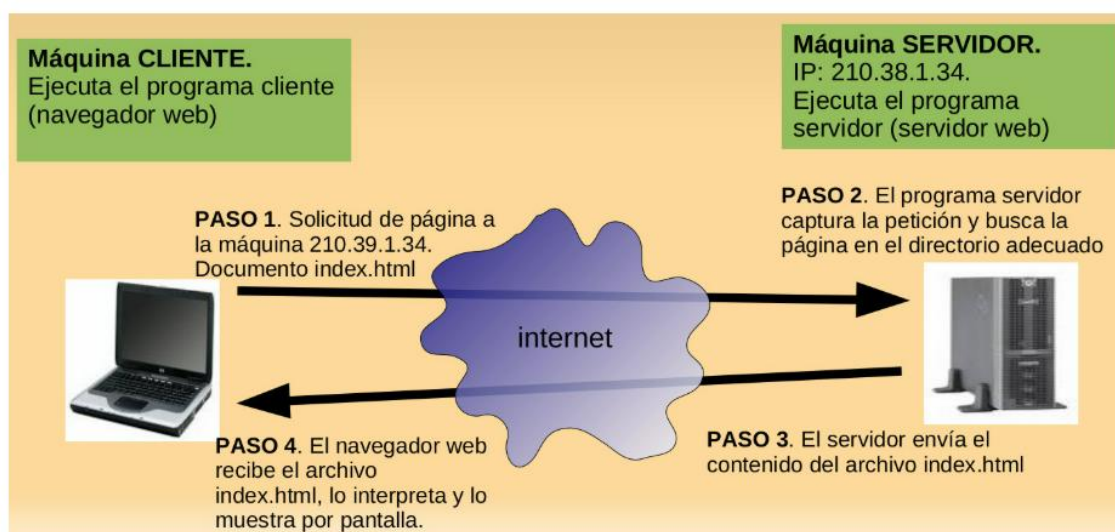
Pues bien, en programación web, nuestro cliente es el **navegador web** (también llamado cliente web).

Y un servidor es cualquier máquina de la red donde se esté ejecutando un programa servidor web como Apache, Tomcat...

2. UNA PETICIÓN WEB EN LA ÉPOCA 1.0

¿Qué era la Web 1.0?

- Se refiere a la primera etapa de la web, entre los años 1990 y principios de los 2000.
- Era estática: los servidores devolvían HTML fijo, sin interacción dinámica.
- Cada vez que el usuario hacía algo, el navegador debía recargar toda la página.



En el esquema, se ilustra lo que ocurre cuando un cliente web (recuerda: tu navegador) envía al servidor la petición de una página estática.

El servidor, en este caso, se limita a enviar al cliente el documento HTML tal cual está almacenado en su disco duro, sin cambiar una sola coma.

¿Cómo era una petición?

1. El cliente (navegador) hace una petición HTTP (GET) a un servidor.
2. El servidor devuelve un archivo HTML ya preparado.
3. Si el usuario quiere más información o hacer algo (por ejemplo, enviar un formulario), el navegador envía otra petición y se recarga la página completa.

Ejemplo típico:

- Usuario accede a `http://tienda.com/productos.php`
- El servidor genera el HTML con PHP y lo devuelve al navegador.
- El navegador muestra el contenido renderizado desde el servidor.

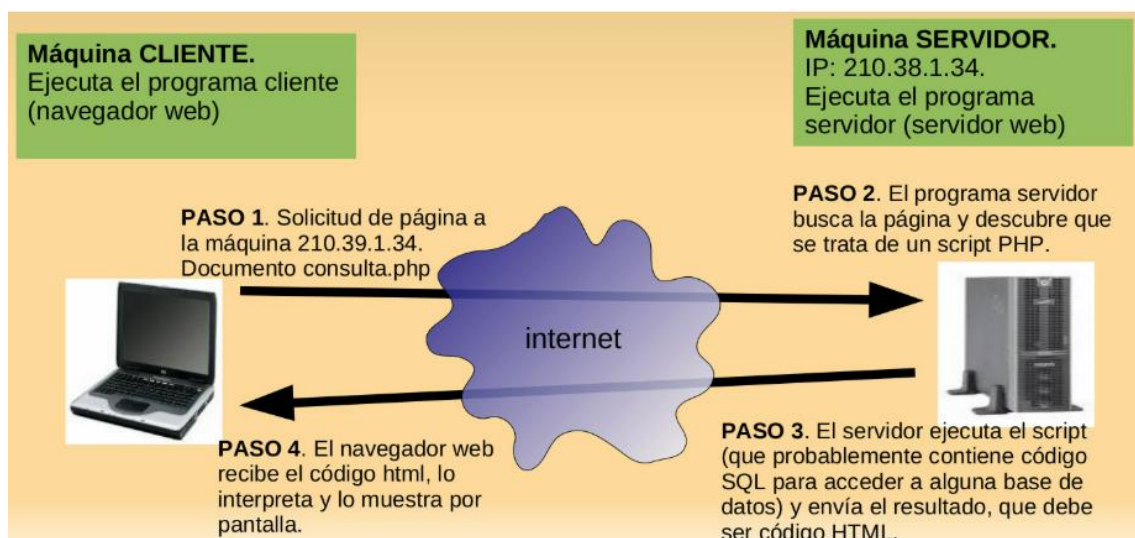
3. UNA PETICIÓN WEB EN LA ÉPOCA 2.0

¿Qué es la Web 2.0?

- Comienza en los años 2004 en adelante.
- Mucho más dinámica e interactiva.
- Aparecen tecnologías como AJAX, JavaScript avanzado, y APIs REST.
- El usuario puede interactuar sin recargar toda la página (ej: Facebook, Gmail...).

¿Cómo es una petición?

1. El cliente (navegador o app) envía peticiones HTTP usando JavaScript (AJAX, `fetch()` o librerías como Axios).
2. El servidor devuelve solo los datos (normalmente en JSON), no una página entera.
3. El navegador procesa los datos y actualiza solo una parte de la página dinámicamente.



Con la web 2.0 la cosa cambia bastante porque aparecen las páginas dinámicas, aunque tendrás que fijarte bien en el esquema para apreciar la diferencia.

Quédate con lo importante: en este esquema, el cliente web no pide un documento HTML, sino *un programa*, que puede estar escrito en PHP o algún otro lenguaje. Eso es lo de menos.

Ese programa se ejecuta en el servidor, y *el resultado de esa ejecución* es lo que recibe el cliente, *no el programa en sí*.

Ejemplo típico:

- Usuario hace clic en "Cargar más productos"
- JS envía un **GET** a **/api/productos?page=2**
- El servidor devuelve:

json

[{ "id": 21, "nombre": "Ratón", "precio": 12.5 }, ...]

- JS lo renderiza directamente sin recargar la página

4. ¿Y LA WEB 3.0?

- No es un estándar oficial, sino un conjunto de ideas y tendencias que están en evolución.
- Propone una web:
 - Más semántica (los datos tienen significado para las máquinas)
 - Descentralizada (sin grandes servidores controladores, uso de blockchain)
 - Con mayor control del usuario sobre su identidad y sus datos
 - Asistida por IA y con personalización más inteligente

Competencias clave en el desarrollo web actual

En el contexto actual del desarrollo multiplataforma, estas son las habilidades y conceptos que se consideran fundamentales para trabajar de forma eficiente y alineada con las prácticas modernas:

1. **Uso estructurado de PHP**, aplicando principios como separación entre lógica y presentación (modelo-vista-controlador o patrones similares).
2. **Diseño y consumo de APIs RESTful**, manejando peticiones HTTP (GET, POST, PUT, DELETE) y respuestas en formato JSON.
3. **Acceso a bases de datos** con seguridad y buenas prácticas, utilizando **PDO** (PHP Data Objects) en lugar de funciones obsoletas.
4. **Gestión de autenticación y autorización** mediante tokens (como **JWT**) y control de **sesiones**.
5. **Programación en JavaScript moderno (ES6+)** para manejar el comportamiento del cliente sin recargar la página.
6. **Uso de herramientas profesionales** como Visual Studio Code, extensiones útiles (PHP Server, REST Client), Git y Postman.
7. **Separación clara entre backend y frontend**, siguiendo una arquitectura orientada a servicios (APIs). Esto significa que debes mantener **separadas las partes del programa que se ejecutan en el servidor (backend) de las que se muestran o ejecutan en el navegador del usuario (frontend)**.
8. **Conocimientos básicos de despliegue**: entorno local (como Docker o PHP Server), y conceptos de publicación web en entornos reales.

5. ¿QUÉ ES UNA SESIÓN?

Cuando un usuario navega por una web, cada vez que hace clic o envía un formulario, el navegador realiza una petición HTTP independiente al servidor.

HTTP es un protocolo sin estado: El servidor no recuerda nada entre una petición y la siguiente.

Por ejemplo:

- El usuario se “logea” en una página.
- En la siguiente página, el servidor no sabe quién es, porque no guarda memoria.

La solución: las sesiones

Una sesión permite que el servidor “recuerde” información sobre un usuario mientras navega por la web. Esa información se almacena en el servidor, no en el navegador.

Elemento	Dónde se guarda	Qué contiene
Sesión	En el servidor	Datos del usuario, su carrito, permisos, etc.
Cookie de sesión (PHPSESSID)	En el navegador	Solo guarda el identificador de la sesión

¿Cómo funciona?

- El servidor crea una sesión y le asigna un ID único (session_id).
- Ese ID se guarda en una cookie en el navegador (PHPSESSID).
- Cada vez que el usuario hace una nueva petición, el navegador envía la cookie con el session_id.
- PHP usa ese ID para recuperar los datos guardados en el servidor.

6. COMO SE UTILIZA UNA SESIÓN EN PHP

1. Iniciar la sesión

- Antes de poder usar sesiones, siempre hay que iniciar con: `session_start();`
- Debe ir antes de cualquier salida (echo, HTML, etc.).

2. Guardar información en la sesión

```
session_start();  
$_SESSION["usuario"] = "Carlos";  
$_SESSION["rol"] = "admin";
```

- Aquí, PHP guarda en el servidor un [array](#) con esos valores asociados a tu sesión

3. Recuperar la información

```
session_start();  
echo "Bienvenido, " . $_SESSION["usuario"];
```

- Esto mostrará: Bienvenido, Carlos

4. Comprobar si una sesión existe

```
session_start();  
if (isset($_SESSION["usuario"])) {  
    echo "Usuario activo: " . $_SESSION["usuario"];  
} else {  
    echo "No has iniciado sesión.";  
}
```

5. Eliminar una sesión (logout)

```
session_start();  
session_unset(); // Elimina las variables  
session_destroy(); // Destruye la sesión
```

7. ¿QUÉ ES EXACTAMENTE \$_SESSION?

- **\$_SESSION** es una variable superglobal de PHP.

Eso significa que está disponible en todos los scripts del programa (sin necesidad de pasarla por parámetros), una vez que la sesión está iniciada con `session_start()`.

- **\$_SESSION** es un array asociativo que guarda pares **clave → valor**, exactamente igual que otros arrays de PHP, pero su contenido se conserva entre distintas peticiones del mismo usuario.
- **\$_SESSION** no se comparte entre diferentes usuarios ni entre navegadores.

Cada cliente tiene su propia sesión aislada, determinada por su identificador único (**session_id()**).